

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionale» Prädikate mit Beispielen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$A \Rightarrow B \Leftrightarrow (\neg A \vee B) \Leftrightarrow (A \wedge B) \vee (\neg A \wedge \neg B) \Leftrightarrow A \vee \neg B$
 $A \Rightarrow B \Leftrightarrow (\neg A \wedge B) \vee (A \wedge \neg B) \Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow C) \Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow C)$

Begriff	Bedeutung
Abtrennungsregel	$(A \wedge (A \Rightarrow B)) \Rightarrow B$
Kommutativität	$(A \wedge B) \Leftrightarrow (B \wedge A)$ $(A \vee B) \Leftrightarrow (B \vee A)$
Assoziativität	$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$ $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
Distributivität	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$ $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
Absorption	$A \vee (A \wedge B) \Leftrightarrow A$ $A \wedge (A \vee B) \Leftrightarrow A$
Idempotenz	$A \vee A = A$ $A \wedge A = A$
Doppelte Negation	$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$
Konstanten	$W = \text{wahr}$ $F = \text{falsch}$
de Morgan	$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$

$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht
 $\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\bar{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

n -Tupel:

$$\bigtimes_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

Regulär	Kontextfrei	Turing-erkennbar
DEA NEA Regex	CFG PDA Palindrome {0^n 1^n n ≥ 0}	Primzahlen {a^n b^n c^n n ≥ 0}

Begriff	Bedeutung
Alphabet	Eine nichtleere endliche Menge Σ heisst Alphabet . Die Elemente von Σ heissen Zeichen .
Wort	Eine Zeichenkette der Länge n ist ein n -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$. Ein Element von Σ^n heisst Wort der Länge n . Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\epsilon \in \Sigma^0 = \{\epsilon\}$ der Länge 0 heisst das leere Wort .
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$(A, u, t, o, S, p, r) = \text{AutoSpr}$ $a^3 b^5 = \text{aaabbbbbb}$ $b^5 a^3 = \text{bbbbbaaa}$
Sprache	Teilmenge $L \subset \Sigma^*$

WORTLÄNGE	
Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^n, a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

Beispiel 1: Wortlänge		
$ \epsilon = 0$	$ 01010 _1 = 2$	$ a^3 b^5 = 8$
$ 01010 _0 = 3$	$ 01010 _3 = 0$	$ (1291)^7 = 7 \cdot 1291 = 7 \cdot 4 = 28$

Beispiel 2: Sprache	
$L = \Sigma^* \subset \Sigma^*$	$L = \emptyset \subset \Sigma^*$, die leere Sprache
$\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$	
$\Sigma = \text{Unicode}$, $J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$	
$M = \text{eine "Maschine"}$, $L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$	

Beobachtungen 1	
1.1. $ w^n = n \cdot w $	
1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$	

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition Tabellenform Zustandsdiagramm von A

$A = (Q, \Sigma, \delta, q_0, F)$

- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Überabzählbar unendlich

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(\delta(q, a_1), a_2), \dots, a_n)$$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

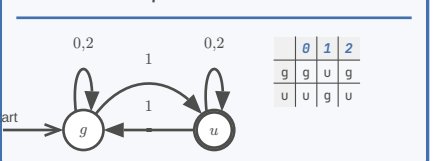
AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert



MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\epsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:
- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\epsilon) = L$
- $F = \{L(w) \in Q \mid \epsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$
Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

w	$L(w)$	Q
ϵ	$L(\epsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid w _0 \text{ ungerade}\}$	q_1
1	$L(1) = \{w \in \Sigma^* \mid w _0 \text{ gerade}\}$	q_0

MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:
1) Akzeptierzustand $\neq$ Nichtakzeptierzustand
2) Iteration: alle unterscheidbaren Paare suchen
3) Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- $|xy| \leq N$
- $|y| > 0$
- $xy^kz \in L \forall k \in \mathbb{N}$

Was zeichnet reguläre Sprachen aus?
- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- $\Rightarrow$ Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
- Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- Wähle ein Wort $w \in L$ mit $|w| \geq N$
- Die Definition **muss** N verwenden!
- Aufteilung des Wortes gemäss Pumping Lemma
- $w = xyz, |xy| \leq N, |y| > 0$
- Auswirkung des Pumpens
- $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^N 1^N$
- Unterteilung $w = xyz$

TODO: reduce confusion

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt
6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

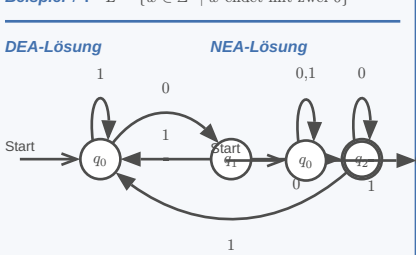
Definition Akzeptieren

$A = (Q, \Sigma, \delta, q_0, F)$
Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei 0}\}$



UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher Automat, PEA)

Start $\rightarrow q_0$ $\xrightarrow{a}$ q_2 $\xrightarrow{b}$ q_0

REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$w \in A \Leftrightarrow f(w) \in B$

Notation: $f : A \leq B$ heisst « A leichter entscheidbar als B »

Entscheidbarkeit

Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar

VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

SATZ VON RICE

LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:
book 273..

KOMPLEXITÄT

Ineffizient: Entscheidbare Probleme mit exorbitant langer Laufzeit sind praktisch «nicht lösbar»

Problemgrösse: Laufzeit hängt von der Problemgrösse n ab

Worst case Laufzeit: Die worst case Laufzeit $t(n)$, die die Maschine zur Verarbeitung von Inputs der Länge n braucht, ist $t(n) = \max\{t(w) \mid |w| \leq n\}$

HARDWAREEINFLUSS

Nichtdeterministische Hardware kann in Zeit $2^{O(t(n))}$ simuliert werden $\Rightarrow$ NTM ist exponentiell schneller

POLYNOMIELLE UND EXPONENTIELLE LAUFZEIT

TODO:
slides 5

Polynomielle Probleme	Exponentielle Probleme
- Gauss-Algorithmus $O(n^3)$ - ...	- Faktorisierung von grossen Zahlen (RSA) - ...
Skalierbarkeit	Sicherheit

Ein Algorithmus hat **polynomielle Laufzeit**, wenn seine Laufzeit durch ein Polynom beschränkt ist: $t(n) = O(n^k)$.

SPRACHEN

Eine Sprache L gehört zur Klasse NP , wenn L von einer nichtdeterministischen TM in **polynomieller Zeit entschieden** werden kann

POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$w \in L \Leftrightarrow V$ akzeptiert $\langle w, c \rangle$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

TODO:
slides 14

POLYNOMIELLE REDUKTION

POLYNOMIELLER LAUFZEIT-VERGLEICH

Seien A und B entscheidbare Sprachen. Eine berechenbare Abbildung

$f : \Sigma^* \rightarrow \Sigma^* : w \mapsto f(w)$

mit
1) $w \in A \Leftrightarrow f(w) \in B$ (Reduktion)
2) $f(w)$ ist berechenbar in polynomieller Zeit in $|w|$
heisst **polynomielle Reduktion** $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

Polynomiell äquivalent: Zwei Sprachen A und B heissen **polynomiell äquivalent**, geschrieben $A \equiv_P B$, wenn $A \leq_P B$ und $B \leq_P A$.

SATZ
Sei $A \leq_P B$. Dann gilt

$B \in P \Rightarrow A \in P, \quad A \notin P \Rightarrow B \notin P$

$B \in NP \Rightarrow A \in NP, \quad A \notin NP \Rightarrow B \notin NP$

TODO:
slides 16,17

REDUKTION VON HAMPATH

TODO:

POLYNOMIELLE AUSFÜLLRÄTSEL

Ein polynomielles Ausfüllrätsel ist eine $n \times m$ -Tabelle, in die Zeichen eines Alphabets Σ eingefüllt werden müssen, so dass als logische Formel ausdrückbare Regeln eingehalten werden, die in polynomieller Zeit ausgewertet werden können. (Beispiel: Sudoku)

SAT: Boolean satisfiability problem: Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

$SAT = \{\varphi \mid \varphi \text{ ist eine erfüllbare logische Formel}\}$

Satz

Jedes polynomielle Ausfüllrätsel A lässt sich polynomiell auf SAT reduzieren.

Variablen

$x_{ijc} = \text{wahr}$

$\Leftrightarrow$ Feld (i, j) der Tabelle enthält Zeichen c $\{\varphi \mid \varphi \text{ erfüllbare logische Formel}\} \Rightarrow \{\varphi \mid \varphi \text{ erfüllbare logische Formel in 3CNF}\}$

Genau ein Zeichen im Feld (i, j)

TODO:

$$\varphi_{ij} = \bigvee_{c \in \Sigma} \left(\underbrace{x_{ijc} \wedge \bigvee_{d \neq c} x_{ijd}}_{c \text{ und kein anderes Zeichen in } (i, j)} \right)$$

Regeln

Spielregeln als Formel φ in Variablen x_{ijc} ausdrücken, die wahr ist genau dann, wenn die Regeln erfüllt sind.

NP-VOLLSTÄNDIGKEIT

Eine entscheidbare Sprache B heisst **NP-vollständig**, wenn sich jede Sprache A in NP polynomiell auf B reduzieren lässt:

$A \leq_P B \quad \forall A \in NP$

NP-vollständige Probleme sind die «schwierigsten» Probleme in NP. Sie sind alle gleich schwierig:

$A, B \text{ NP-vollständig} \Rightarrow (A \leq_P B \wedge B \leq_P A \Leftrightarrow A \equiv_P B)$

Reduktionsprinzip

$A \text{ NP-vollständig} \wedge B \in NP \wedge A \leq_P B \Rightarrow B \text{ NP-vollständig}$

TODO:
slides 2

Wenn ein Problem NP-vollständig ist:

- Lösung braucht typischerweise exponentielle Zeit
- Korrektheit der Lösung ist in polynomieller Zeit zu prüfen

Umgang mit NP-Vollständigkeit in der Praxis:

- Spezialfälle ausnützen
- Problem modifizieren
- Approximation
- Zufall oder Heuristik
- Genetische Algorithmen und ML

Beispiele:

- SAT / 3SAT
- K-VERTEX-COLORING

COOK-LEVIN

3-SAT: Jede Klausel der Normalform hat maximal 3 Literale.

Satz

SAT ist NP-vollständig.

Das bedeutet:

Sei A eine Sprache in NP

$\Rightarrow$ Es gibt eine nichtdeterministische TM M , die A in polynomieller Zeit $O(t(n))$ entscheidet

$\Rightarrow A$ kann polynomiell auf SAT reduziert werden: $A \leq_P SAT$

Vorgehensweise

Beschreibe das Finden der Berechnungsgeschichte von M als ein polynomielles Ausfüllrätsel

TODO:
slides 5

TODO:
slides 7

«Ausfüllrätsel» in diesem Fall die Berechnungsgeschichte der TM.

Überprüfe, ob alle Zustandsübergänge valid sind.

TODO:
book 328-343/368

$SAT \leq_P 3SAT$

TODO:
 $\rightarrow$ Erfüllungsäquivalenz

Beweis von Cook-Levin

- Jedes Problem lässt sich auf eine Formel reduzieren
- Die Reduktion ist polynomiell
- Die Formel wird aus der Berechnungsgeschichte abgeleitet
- Die Formel kann in CNF konstruiert werden

Erfüllungsäquivalente Umformung

TODO:
slides 11

ÖFFENTLICHES SCHLÜSSELSYSTEM

TODO:
slides 15

KARP-KATALOG

TODO:
slides 16